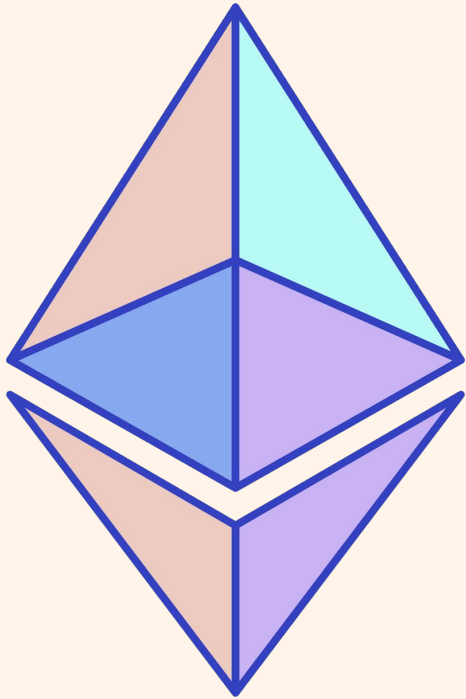
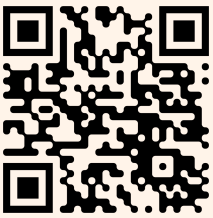


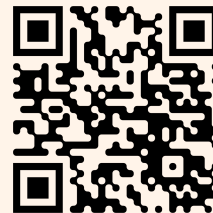


Safely Snarkifying the Ethereum Protocol

Derek Sorensen
Ethereum Foundation Research

2 April 2026

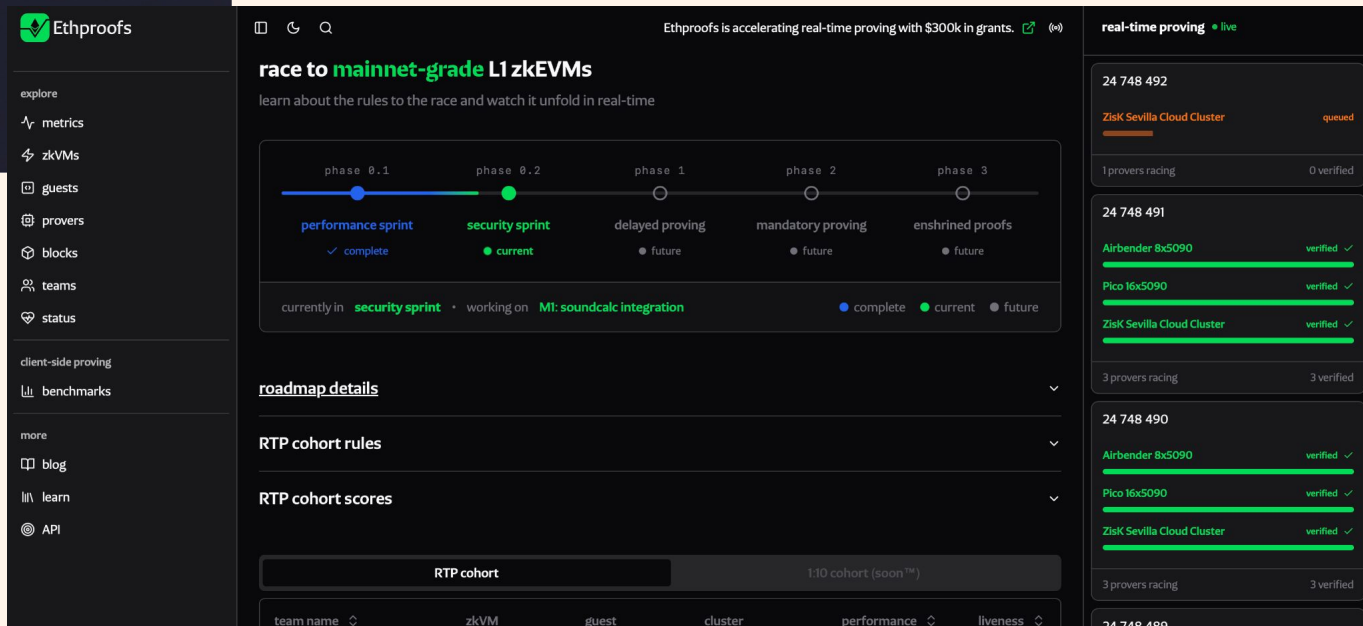
SNARKS, zkVMs, and Ethereum



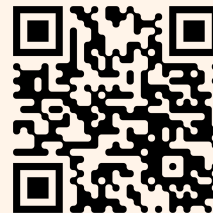
lean cryptography

Hash-based cryptography is emerging as the ideal foundation for reshaping the ecosystem:

- the explosive **rise of SNARKs**
- the looming **quantum threat**



SNARKS, zkVMs, and Ethereum

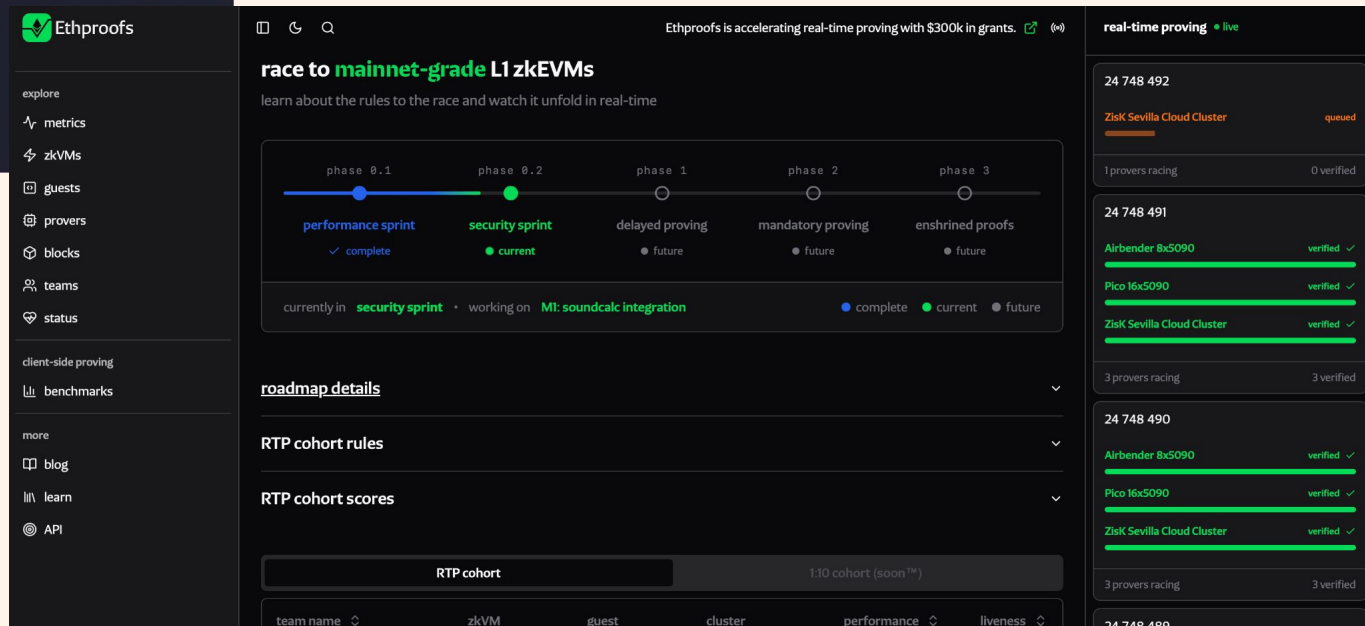


lean cryptography

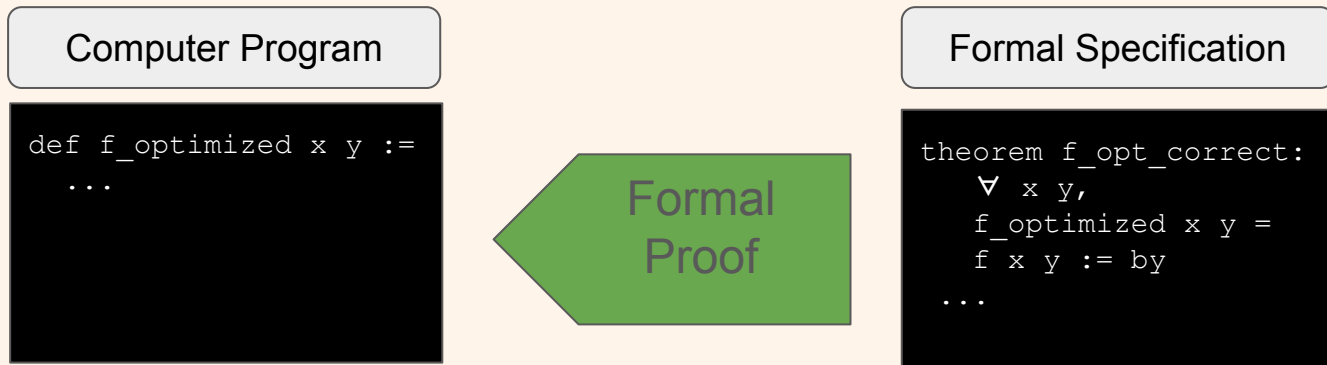
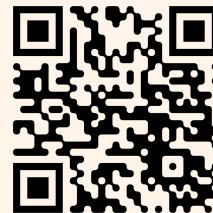
Hash-based cryptography is emerging as the ideal foundation for reshaping the ecosystem:

- the explosive **rise of SNARKs**
- the looming **quantum threat**

SNARKS are critical infrastructure.

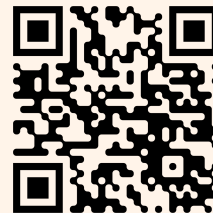


Formal Verification for High Assurance



The program is proved **correct** with regards to the **description in the formal specification**.*

Protocol Snarkification



“... obtain the highest possible level of assurance for zkEVMs, the end goal being bug-free zkEVMs.”

RISC-V zkVM Track

Verify the arithmetization of RISC-V zkVMs. Ensure polynomial constraints exactly encode RISC-V instruction semantics based on the official Sail RISC-V specification.

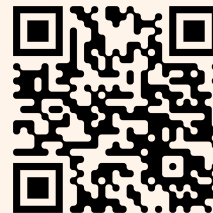
EVM Track

Show that an EVM running on RISC-V correctly implements the EVM specification. Focus on performant implementations compilable to RISC-V zkVMs.

Cryptography Track

Verify the specification, proofs of security, and implementation of proof systems and cryptographic primitives used by zkVMs.

Protocol Snarkification



“... obtain the highest possible level of assurance for zkEVMs, the end goal being bug-free zkEVMs.”

RISC-V zkVM Track

Verify the arithmetization of RISC-V zkVMs. Ensure polynomial constraints exactly encode RISC-V instruction semantics based on the official Sail RISC-V specification.

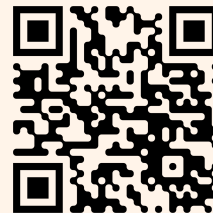
EVM Track

Show that an EVM running on RISC-V correctly implements the EVM specification. Focus on performant implementations compilable to RISC-V zkVMs.

Cryptography Track

Verify the specification, proofs of security, and implementation of proof systems and cryptographic primitives used by zkVMs.

Protocol Snarkification



“... obtain the highest possible level of assurance for zkEVMs, the end goal being bug-free zkEVMs.”

RISC-V zkVM Track

Verify the arithmetization of RISC-V zkVMs. Ensure polynomial constraints exactly encode RISC-V instruction semantics based on the official Sail RISC-V specification.

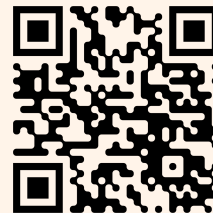
EVM Track

Show that an EVM running on RISC-V correctly implements the EVM specification. Focus on performant implementations compilable to RISC-V zkVMs.

Cryptography Track

Verify the specification, proofs of security, and implementation of proof systems and cryptographic primitives used by zkVMs.

Protocol Snarkification



“... obtain the highest possible level of assurance for zkEVMs, the end goal being bug-free zkEVMs.”

RISC-V zkVM Track

Verify the arithmetization of RISC-V zkVMs. Ensure polynomial constraints exactly encode RISC-V instruction semantics based on the official Sail RISC-V specification.

EVM Track

Show that an EVM running on RISC-V correctly implements the EVM specification. Focus on performant implementations compilable to RISC-V zkVMs.

Cryptography Track

Verify the specification, proofs of security, and implementation of proof systems and cryptographic primitives used by zkVMs.



VCV-io

Formalized Cryptography Proofs in Lean 4

● Lean ☆ 76 🍷 16

oracle computations, underlies
core IOR framework of ArkLib

ArkLib

Formally Verified Arguments of Knowledge in Lean

● Lean ☆ 182 🍷 47

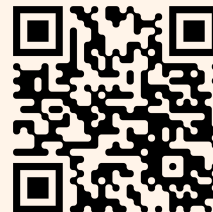
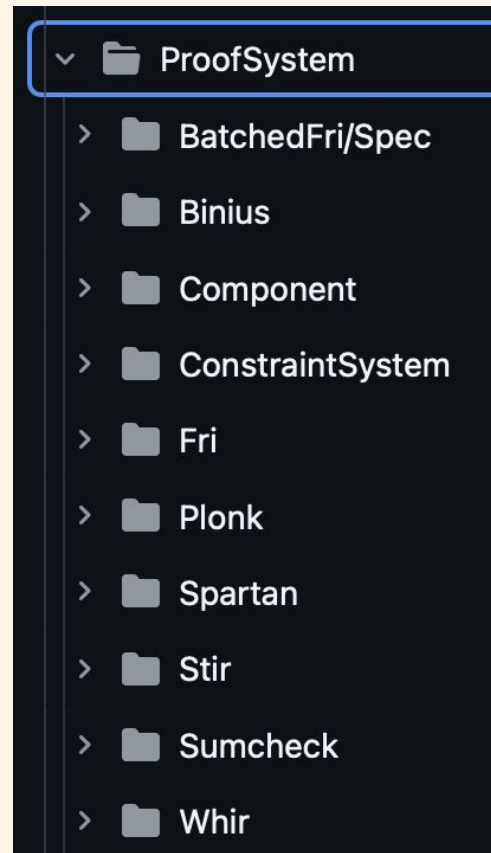
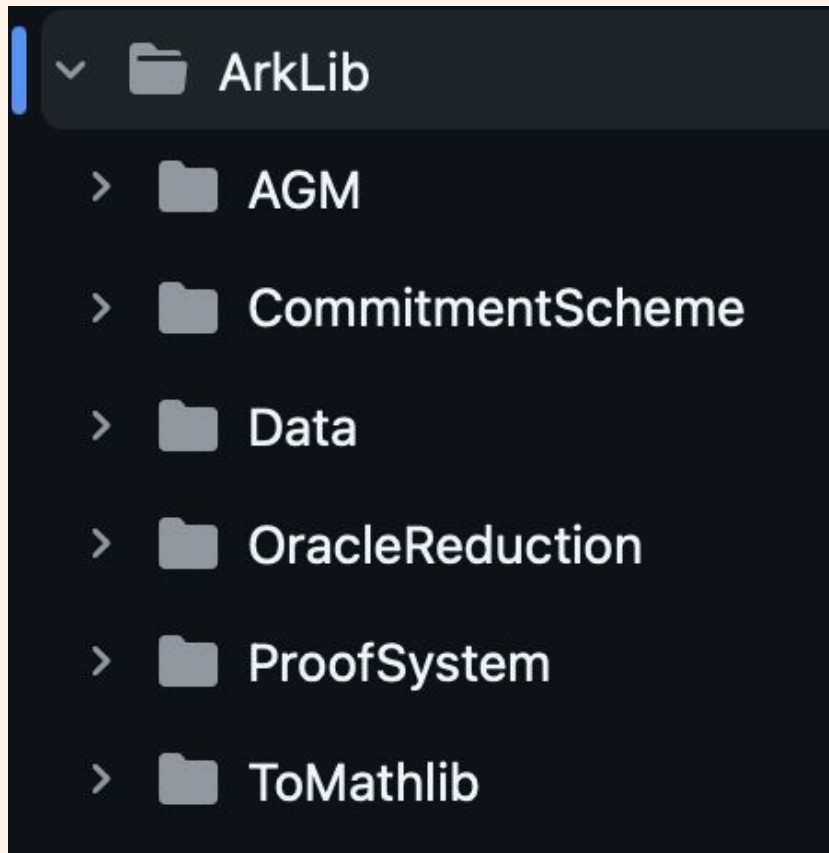
SNARKS and proof systems.

CompPoly

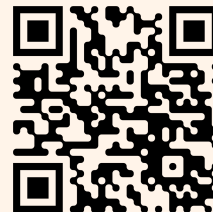
A computable model of Polynomials in Lean.

● Lean ☆ 33 🍷 14

Computable polynomials and
finite fields.



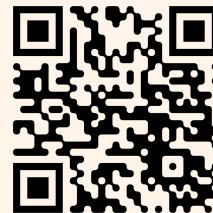
Round Consistency in FRI (Arklib)



```
/--
Completeness of the round consistency check.

Given a polynomial `f`, challenge `γ`, and `n`-th roots of unity `ω`, when `f` is honestly
evaluated at the scaled points `{ω i * s_0}`, the round consistency check succeeds with the
value `(foldNth n f γ).eval (s_0^n)`. This establishes that the Lagrange interpolant through
the evaluation points matches the n-way folding operation at the challenge point.
-/
lemma generalised_round_consistency_completeness
  {F : Type} [inst1 : Field F] [DecidableEq F] {f : Polynomial F}
  {n : ℕ} [inst : NeZero n]
  {γ : F}
  {s_0 : F}
  {ω : Fin n → F}
  (h : ∀ i, (ω i) ^ n = 1)
  (h1 : s_0 ≠ 0)
  :
    roundConsistencyCheck
      γ
      (List.map (fun i => (ω i * s_0, f.eval (ω i * s_0))) (List.finRange n))
      ((foldNth n f γ).eval (s_0^n)) = true := by
  unfold roundConsistencyCheck
  simp only [List.get_eq_getElem, List.getElem_map, List.getElem_finRange, Fin.cast_mk,
    beq_iff_eq]
  unfold foldNth
  conv =>
  left
  rw [splitNth_def n f]
  rw [Polynomial.eval_finset_sum]
  simp only [eval_mul, eval_C, eval_pow]
  have eval_eval2_pow_eq_eval_pow {s : F} (i) :
    eval s (eval2 C (X ^ n) (splitNth f n i)) = (splitNth f n i).eval (s ^ n) := by
```

zkVM Verification Pipeline



Computer Program

Formal Specification



zkVMs implemented in
Rust

Formal
Proof

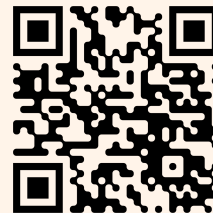
ArkLib

Formally Verified Arguments of Knowledge in Lean

● Lean ☆ 182 🗨️ 47

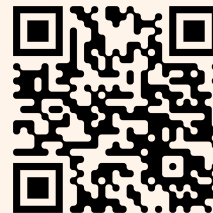
LEAN

Protocol Snarkification



*“... obtain the highest possible level of assurance for zkEVMs,
the end goal being bug-free zkEVMs.” **

Protocol Snarkification



*“... obtain the highest possible level of assurance for zkEVMs, the end goal being bug-free zkEVMs.” **

** modulo ...*

- *the verification boundary*
- *bugs in specification*
- *bugs in proof tools*
- *...*

